Introduction to Optimization

Prof. Karl Ezra Pilario

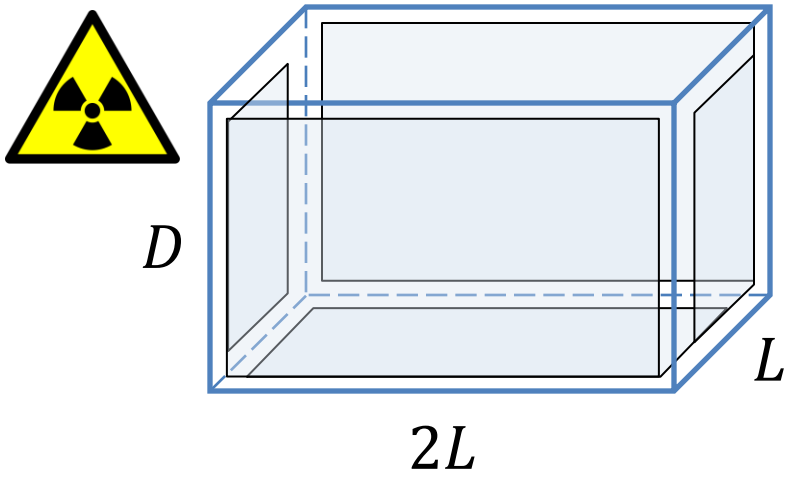
EgyE 332

1s AY 2026-2027

Outline

1. Motivational Example
2. Optimization Problems
3. Why is Optimization hard?
4. Overview of optimization solvers
5. Introduction to Pyomo

Why Optimization? A Motivational Example



Design a large rectangular storage pit which must hold exactly 8 m^3 of *radioactive material*.

The storage floor and walls must be lined with lead which costs PhP 1,200 per square meter.

Find the optimal length (L) and depth (D) that minimizes the cost of lead.

Objective: Minimize the cost of lead

$$\text{Cost}(D, L) = 1200(6DL + 2L^2)$$

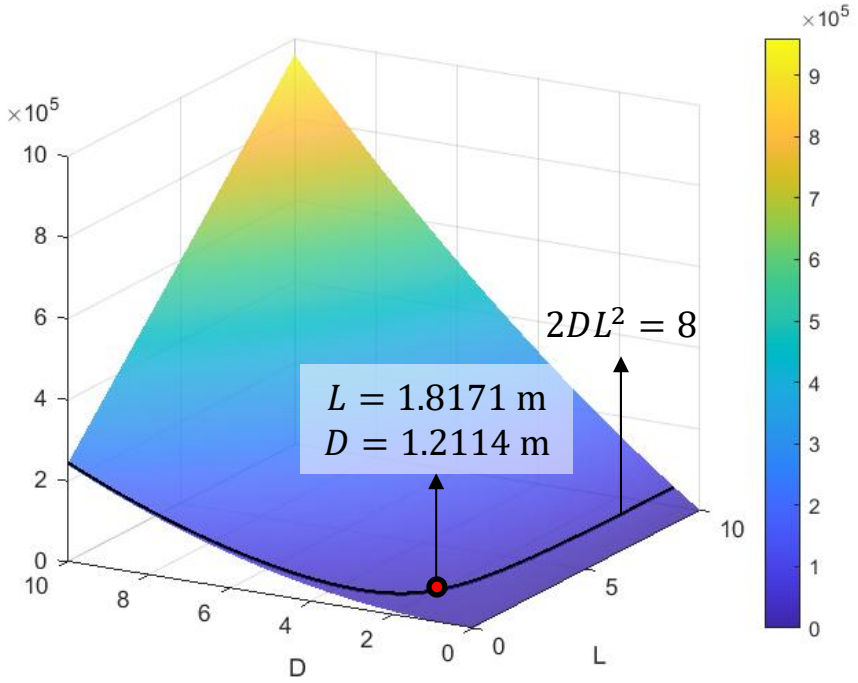
Constraint: Total volume must be 8 m^3

$$2DL^2 = 8$$

Answer:

$$\text{Optimal Cost} = \text{PhP } 23{,}773.88$$

$$\text{Cost}(1\text{ m}, 2\text{ m}) = \text{PhP } 24{,}000.00$$



Why Optimization? A Motivational Example

This example is simple enough to be analytically solved:

Objective: $\min 1200(6DL + 2L^2)$

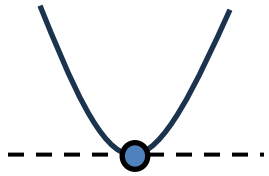
Constraint: $2DL^2 = 8$

Isolate D in the constraint: $D = \frac{4}{L^2}$

Substitute in the objective: $\min 1200\left(6 \cdot \frac{4}{L^2} \cdot L + 2L^2\right)$

We can **drop** the 1200: $\min \frac{24}{L} + 2L^2$

Differentiate and **equate to 0**: $f(L) = \frac{24}{L} + 2L^2$

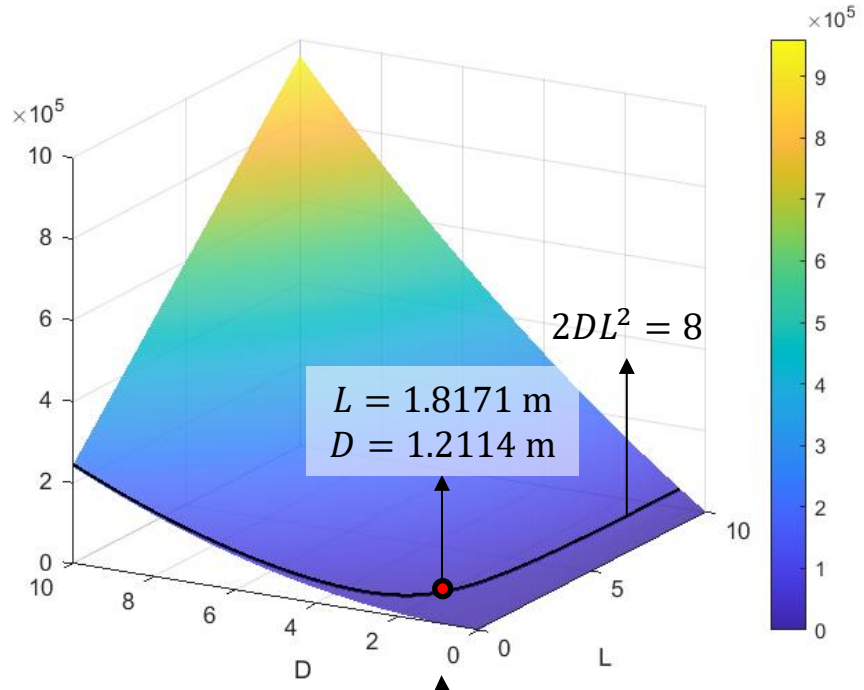


$$f'(L) = -\frac{24}{L^2} + 4L = 0$$

$$4L^3 = 24$$

$$L = \sqrt[3]{6} = \boxed{1.82} \qquad D = \frac{4}{\left(\sqrt[3]{6}\right)^2} = \boxed{1.21}$$

But what if we have arbitrarily *many* constraints (both equalities and inequalities) and arbitrarily *many* variables?



Optimization Problems

In general, all optimization problems are of the form:

Min / Max:	$f(\mathbf{x})$	Objective function
Subject to:	$\mathbf{h}(\mathbf{x}) = \mathbf{0}$	Equality constraints
	$\mathbf{g}(\mathbf{x}) \leq \mathbf{0}$	Inequality constraints
	$\mathbf{x}^{\mathbf{L}} \leq \mathbf{x} \leq \mathbf{x}^{\mathbf{U}}$	Variable bounds

Objective: Minimize the cost of lead

$$\text{Cost}(D, L) = 1200(6DL + 2L^2)$$

Constraint: Total volume must be 8 m^3

$$2DL^2 = 8$$

Answer: $\text{Optimal Cost} = \text{PhP } 23{,}773.88$

$$L = 1.8171 \text{ m}$$
$$D = 1.2114 \text{ m}$$

Vectors: Decision Variables

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \end{bmatrix} \quad \mathbf{h}(\mathbf{x}) = \begin{bmatrix} h_1(x_1, x_2, \dots) \\ h_2(x_1, x_2, \dots) \\ h_3(x_1, x_2, \dots) \\ \vdots \end{bmatrix} \quad \mathbf{g}(\mathbf{x}) = \begin{bmatrix} g_1(x_1, x_2, \dots) \\ g_2(x_1, x_2, \dots) \\ g_3(x_1, x_2, \dots) \\ \vdots \end{bmatrix} \quad \mathbf{0} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ \vdots \end{bmatrix}$$

Optimization Problems

Having an objective function does *not necessarily mean* that we must optimize right away.

Let:
N = no. of decision variables (unknowns)
M = no. of independent equality constraints (equations)

$$\min \ 1200(6DL + 2L^2)$$
$$s.t. \ \ 2DL^2 = 8$$

N > M

Underdetermined

- More unknowns than equations
- Many solutions are *feasible*, but only one (or some) of them is the BEST—this is when we optimize!

$$\min \ 1200(6DL + 2L^2)$$
$$s.t. \ \ 2DL^2 = 8$$
$$2D = L$$

N = M

Just-determined

- No. of unknowns = No. of equations
- The solution/s can be exactly solved as a system of N unknowns, N equations.
- Optimization is NOT needed.

$$\min \ 1200(6DL + 2L^2)$$
$$s.t. \ \ 2DL^2 = 8$$
$$2D = L$$
$$L - D = 1$$

N < M

Overdetermined

- More equations than unknowns
- There may be no solution that satisfies ALL constraints simultaneously.
- Optimization is NOT needed.

Optimization Problems

Operations research is the umbrella term.

Operations Research
a branch of mathematics concerned with the application of scientific methods and techniques to **decision-making problems** and with establishing the best or **optimal solutions** (Rao, 2009).

- Decision Analysis
- Simulation
- Optimization
- System Dynamics
- Queueing Theory
- Forecasting
- Reinforcement Learning

Can pose problems under ↓

Optimization
The act of obtaining the best result under given circumstances (Rao, 2009).

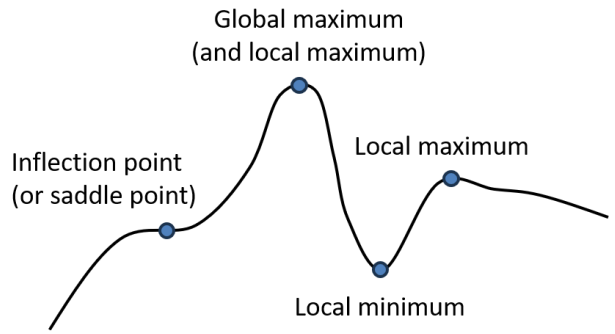
Can be described technically by ↓

Mathematical Programming
Deals with the **formulation**, **solution**, and **analysis** of optimization problems (El-Halwagi, 2012).

- LP
- IP
- MILP
- NLP
- MINLP

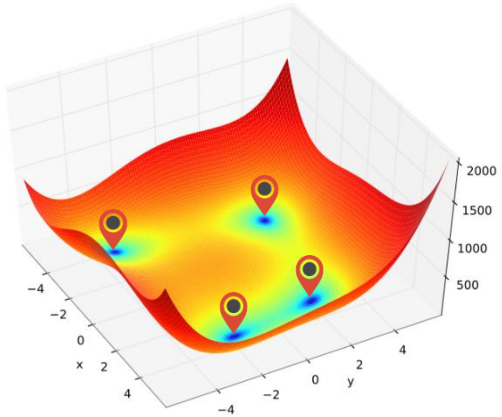
Why is Optimization hard?

- **Objective Landscape**
- Availability of Information
- Size / Complexity
- Nature of Constraints
- Solution and Implementation



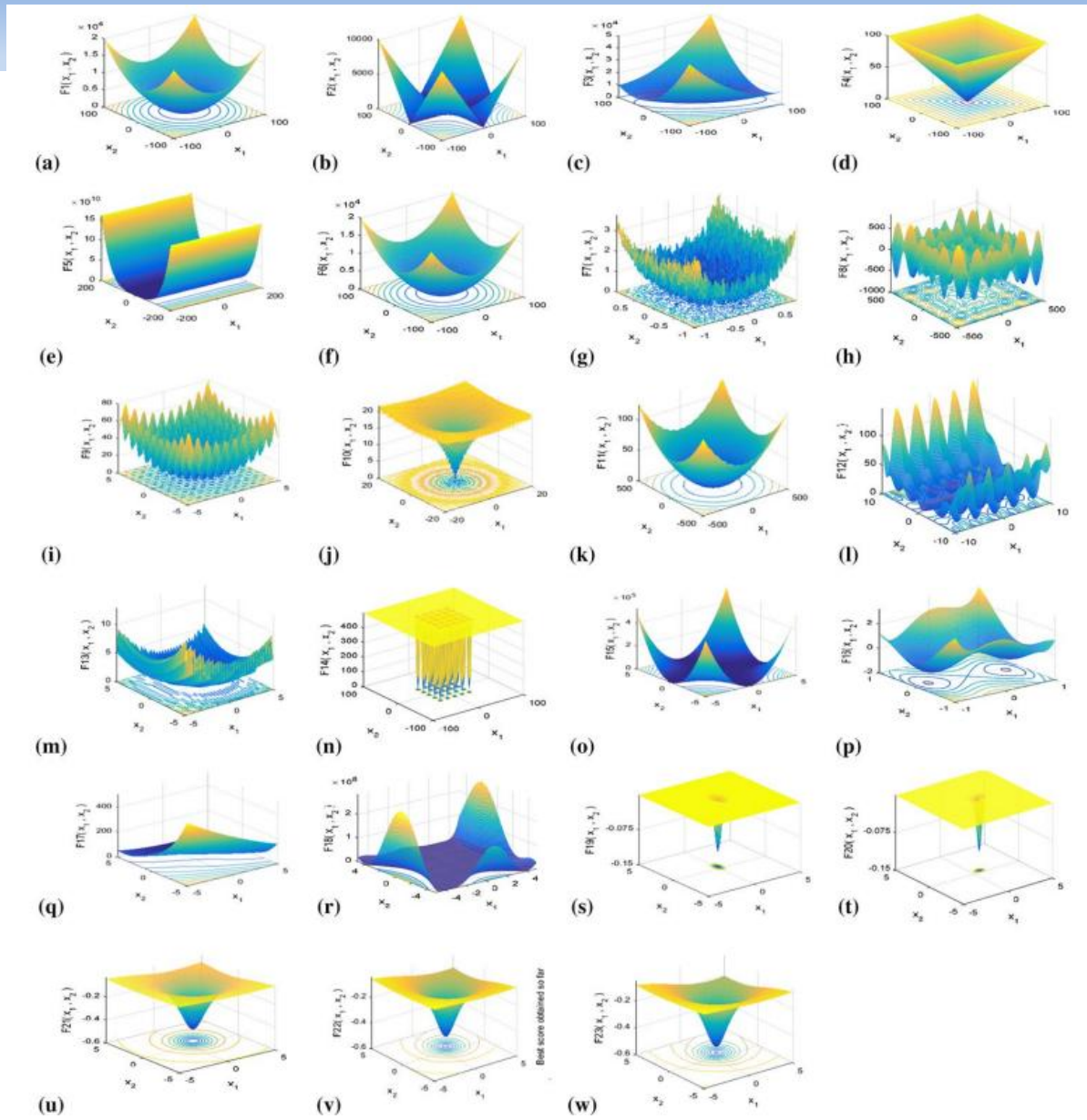
Objective functions can:

- Be *multi-modal (multiple local minima exist)*
- Be *non-differentiable*
- Be *non-smooth*
- Have *flat landscapes*
- Have *saddle points*
- Have *discontinuities*
- Be *multi-objective*



Minimize: $(x_1^2 + x_2 - 11)^2 + (x_1 + x_2^2 - 7)^2$

Subject to: $-5 \leq x_1 \leq 5; \quad -5 \leq x_2 \leq 5$



Fakhouri, H.N., Hamad, F. & Alawamrah, A. Success history intelligent optimizer. *J Supercomput* **78**, 6461–6502 (2022). https://doi.org/10.1007/s11227-021-04093-9

Why is Optimization hard?

Objective Landscape

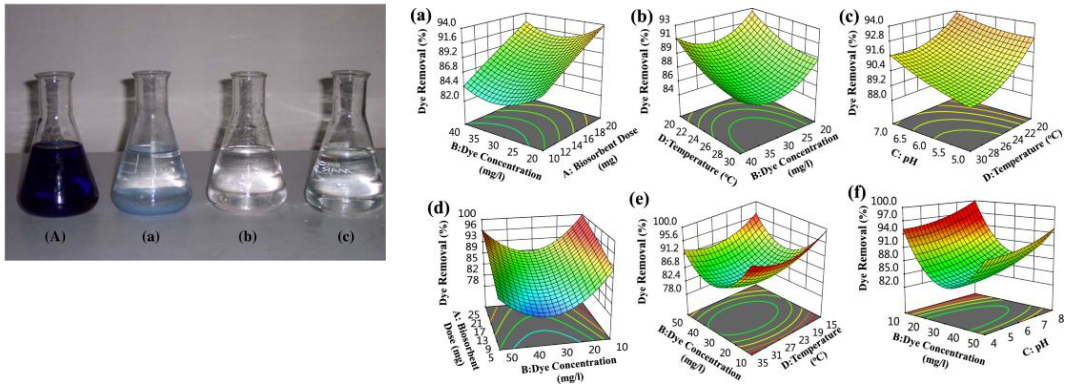
Availability of Information

Objective functions can:

- Be *multi-modal (multiple local minima exist)*
- Be *non-differentiable*
- Be *non-smooth*
- Have *flat landscapes*
- Have *saddle points*
- Have *discontinuities*
- Be *multi-objective*
- **Be *expensive* to sample**
- **Have some degree of *uncertainty***

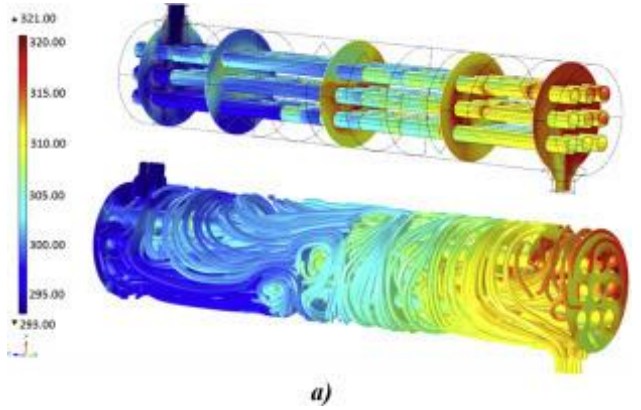
Some goals (objectives) don't even have a known mathematical expression. They're called ***black-box** objectives*. We can only *sample* it a few times.

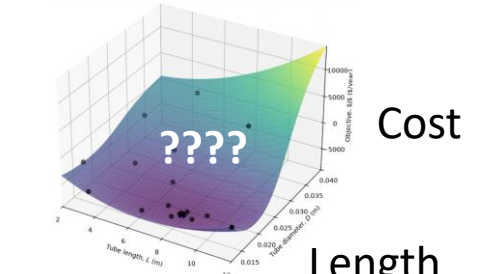
We can use Design of Experiments (DoE) to "sample" our objective at strategic conditions. For example, wastewater treatment experiments:



* Response surface methodology (RSM) is one method to optimize experimental conditions, but it commonly assumes an ANOVA model only.

Some objectives require a *complex simulation* that may take days to render to get even one sample. For example, CFD simulations:





Why is Optimization hard?

Nature of Constraints

Feasible solution

A set of values of variables that satisfy all constraints to the desired degree of precision.

$$\boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \end{bmatrix}$$

Local minimum

A feasible solution, $\boldsymbol{x}^*$, such that $f(\boldsymbol{x}^*) \leq f(\boldsymbol{x})$ for all $\boldsymbol{x}$ in a *small neighborhood* around $\boldsymbol{x}^*$.

Global minimum

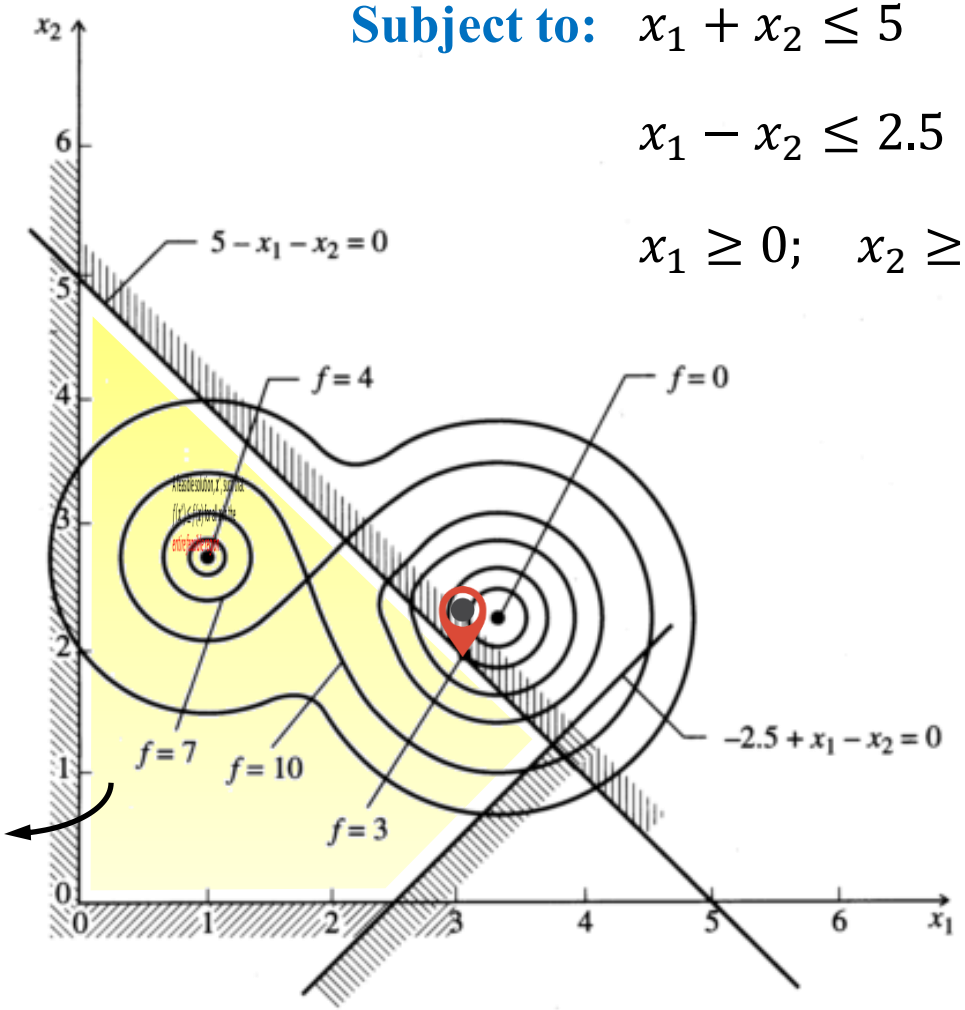
A feasible solution, $\boldsymbol{x}^*$, such that $f(\boldsymbol{x}^*) \leq f(\boldsymbol{x})$ for all $\boldsymbol{x}$ in the *entire feasible region*.

Minimize: $f = (x_1 - 3)^2 + (x_2 - 4)^2$

Subject to: $x_1 + x_2 \leq 5$

$x_1 - x_2 \leq 2.5$

$x_1 \geq 0; \quad x_2 \geq 0$



Feasible Region
The set of all feasible solutions.

Source: Edgar, T., Himmelblau, L., and Lasdon, L., 2003. *Optimization of Chemical Processes*. 2nd Ed. New York: McGraw-Hill.

Why is Optimization hard?

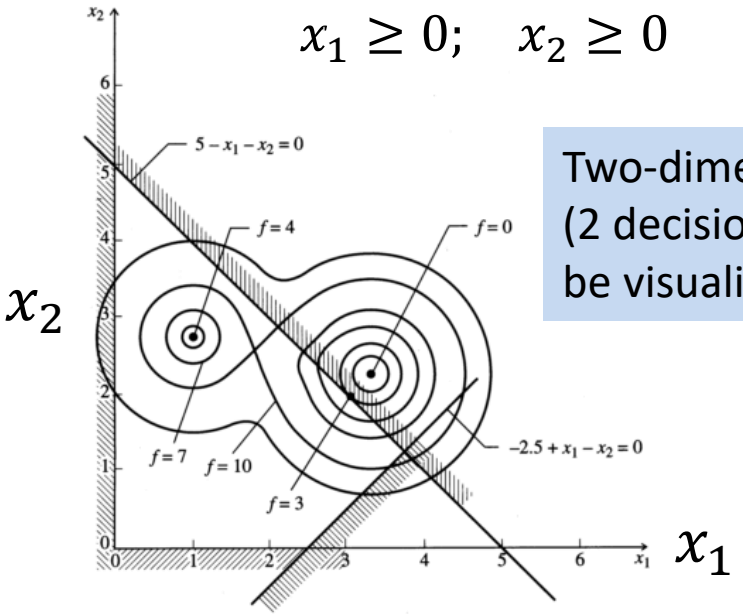
Size / Complexity

Minimize: $f = (x_1 - 3)^2 + (x_2 - 4)^2$

Subject to: $x_1 + x_2 \leq 5$

$x_1 - x_2 \leq 2.5$

$x_1 \geq 0; \quad x_2 \geq 0$



Two-dimensional problems (2 decision variables) can be visualized.

Minimize: $f = \sum_{i=1}^{100} x_i Cost_i$

Subject to: $\sum_i x_i = 1$

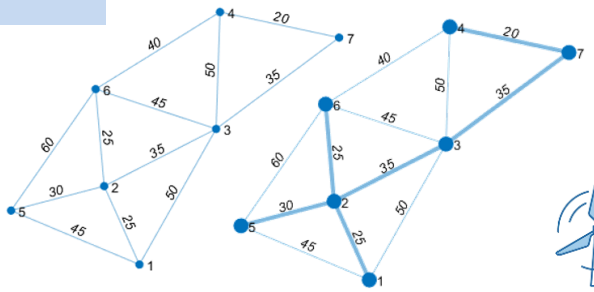
$x_i \geq 0$

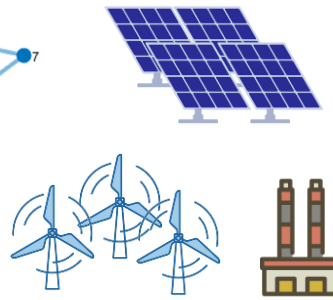
High-dimensional problems cannot be visualized.

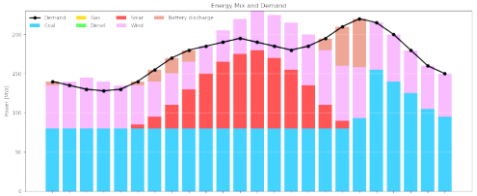
Discrete decisions make the optimization *combinatorial*.

Some decision variables can be:

Binary	**Integer**	**Continuous**
$x_i \in \{0,1\}$	$x_i \in \{1,2,3 \dots\}$	$x_i \in [0,100]$







Energy Mix

Why is Optimization hard?

Size / Complexity

1. Static problems

Single-period / Spatial: $\min f(x)$

Multi-period (Temporally decoupled): $\min \sum_{t=1}^{T} f_t(x_t)$

2. Dynamic problems

Discrete-time (Temporally coupled): $\min \sum_{t=1}^{T} f_t(x_t, s_t)$

$$s_{t+1} = F_t(x_t, s_t)$$

Continuous-time: $\min \sum_{t=1}^{T} f_t(x_t, s_t) + \int_{t_i}^{t_f} L(x_t, u_t, s_t)\, dt$

$$\frac{ds_t}{dt} = F_t(x_t, u_t, s_t, t)$$

Examples:

- Single-period Optimal Energy Mix
- Single-period Economic Dispatch
- Multi-period Economic Dispatch
- Spatial Network Planning
- Fuel Blending

- Unit commitment with ramping
- Reservoir scheduling
- Battery scheduling with state-of-charge
- Inventory management

- *Optimal Control:* Wind Farm, Battery, Fuel Cells, Building Energy / HVAC
- Optimal Thermal Management
- Geological Carbon Storage
- Optimal Oil-Well Production

Why is Optimization hard?

Solution and Implementation

In general, the method for solving mathematical models is highly dependent on the **type of model**:

TYPE	Objective Function	Constraints	Decision Variables
Linear Program (**LP**)	Linear	Linear	Continuous
Nonlinear Program (**NLP**)	At least the objective function or 1 constraint is nonlinear		Continuous
Integer Program (**IP**)	Linear or nonlinear	Linear or nonlinear	All Integer
Mixed-integer Linear Program (**MILP**)	Linear	Linear	Some Integer; Others Continuous
Mixed-integer Nonlinear Program (**MINLP**)	At least the objective function or 1 constraint is nonlinear		Some Integer; Others Continuous

Other problems:

- Poor scaling / numerical issues
- Changing data / environments
- Model-reality mismatch
- Computational burden
- Practical adoption issues / social resistance

Source: Chap.14; El-Halwagi, M., 2012. *Sustainable Design Through Process Integration.* USA: Elsevier Inc.

*The word "program" is due to George Dantzig (1974).

How to Solve Optimization Problems

In general, the method for solving mathematical models is highly dependent on the **type of model**:

For Linear Programming:

1. Simplex Method
2. Lagrange Multipliers

For Nonlinear Programming:

1. Lagrange Multipliers
2. Interior-Point Optimizer (IPOPT)
3. Generalized Reduced Gradient (GRG)
4. Gradient Descent (BFGS, Conjugate Gradient)

For Integer Programming:

1. Branch-and-bound
2. Cutting-plane methods
3. Branch-and-cut

For Mixed-Integer Linear / Nonlinear Programming:

Deterministic/Exact

1. Branch-and-bound
2. Cutting plane methods
3. Inner/Outer Approximation
4. Interval Arithmetic
5. Convex solvers with Multi-start

Stochastic/Metaheuristic

1. Simulated Annealing (SA)
2. Swarm Intelligence
   a. Particle swarm optimization
   b. Ant/Bee colony optimization
3. Evolutionary Algorithms
   a. Genetic Algorithm (GA)
   b. Differential Evolution

How to Solve Optimization Problems

Zero-order Methods

Derivative-free, black-box

- Grid Search / Exhaustive Search
- Random Search

- Metaheuristic Search
  - Genetic Algorithms
  - Particle Swarm
  - Simulated Annealing
  - Differential Evolution
  - CMAES, etc.

- Bayesian Optimization / Surrogate-based

First-order Methods

Uses 1st derivative information

- Steepest Descent or Gradient Descent
- Stochastic Gradient Descent (SGD)
  - RMSProp, Adam, etc.
- Conjugate Gradient methods (e.g. Fletcher-Reeves)
- Coordinate Descent
- Generalized Reduced Gradient (GRG)

Second-order Methods

Uses 1st and 2nd derivative information

- *Newton's method*
- *Quasi-newton method (BFGS)*

- Levenberg-Marquardt
- Trust Region Newton Methods
- Active Set Methods
- Interior-Point Methods (IPOPT)
- Sequential Quadratic Programming (SQP)

How to Solve Optimization Problems

MATLAB Optimization Toolbox Solvers:



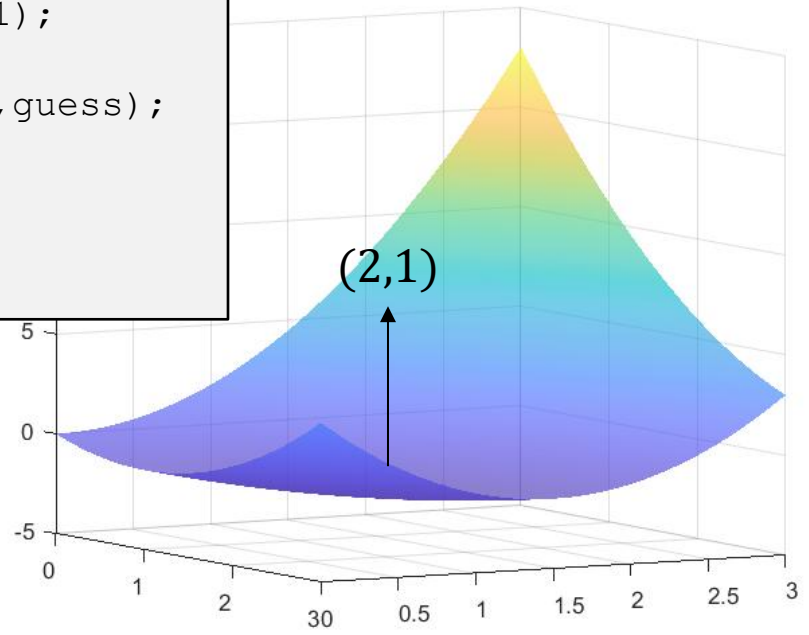
MATLAB

1. `bintprog` (Binary integer programming)
2. `fgoalattain` (Multi-objective goal attainment)
3. `fminbnd` (Single-variable nonlinear minimization with bounds)
4. `fmincon` (Constrained nonlinear minimization)
5. `fminimax` (Minimax optimization)
6. `fminsearch` (Unconstrained nonlinear minimization)
7. `fminunc` (Unconstrained nonlinear minimization)
8. `fseminf` (Semi-infinite minimization)
9. `fsolve` (Nonlinear equation solving)
10. `fzero` (Single-variable nonlinear equation solving)
11. `ga` (Genetic Algorithm)
12. `gamultiobj` (Multiobjective optimization using Genetic Algorithm)
13. `linprog` (Linear programming)
14. `lsqcurvefit` (Nonlinear curve fitting)
15. `lsqnonlin` (Constrained linear least squares)
16. `lsqnonneg` (Nonnegative linear least squares)
17. `patternsearch` (Pattern search)
18. `quadprog` (Quadratic programming)
19. `simmulannealbnd` (Simulated annealing algorithm)

```
>> help fminsearch
 FMINSEARCH Multidimensional unconstrained nonlinear
        minimization (Nelder-Mead).
```

Example: Minimize $f(x_1, x_2) = x_1^2 + 2x_2^2 - 2x_1x_2 - 2x_1$

```
>> f = @(x) x(1).^2 + 2*x(2).^2 ...
    - 2*x(1).*x(2) - 2*x(1);
>> guess = [0 0];
>> [x,val] = fminsearch(f,guess);
x =
    2.0000    1.0000
val =
   -2.0000
```



Introduction to Pyomo



https://www.pyomo.org/

Pyomo

is a Python-based open-source software package that supports a diverse set of optimization capabilities for formulating, solving, and analyzing **optimization models**.

Developed by:

Sandia National Laboratories

COIN-OR

Example 1:

$$\max \quad 30x_1 + 40x_2$$

$$s.t. \quad 3x_1 + x_2 \leq 180$$
$$2x_1 + 3x_2 \leq 240$$
$$x_1, x_2 \geq 0$$

Installation:

```
pip install pyomo
conda install -c conda-forge pyomo
```

Solvers:

```
conda install -c conda-forge glpk
conda install -c conda-forge ipopt
```

```python
import pyomo.environ as pyo
import pandas as pd
import numpy as np

# Create the optimization model
model = pyo.ConcreteModel()

# Decision variables
model.x1 = pyo.Var(domain=pyo.NonNegativeReals)
model.x2 = pyo.Var(domain=pyo.NonNegativeReals)

# Objective Function
model.obj = pyo.Objective(expr=30*model.x1 + 40*model.x2,
    sense=pyo.maximize)

# Constraints
model.con1 = pyo.Constraint(expr=3*model.x1 + model.x2 <= 180)
model.con2 = pyo.Constraint(expr=2*model.x1 + 3*model.x2 <= 240)

solver = pyo.SolverFactory("glpk")
results = solver.solve(model, tee=False)

print("Solver status:", results.solver.status)
print("Termination condition:", results.solver.termination_condition)

print("Optimal solution:")
print(f"x1 = {pyo.value(model.x1)}")
print(f"x2 = {pyo.value(model.x2)}")
```

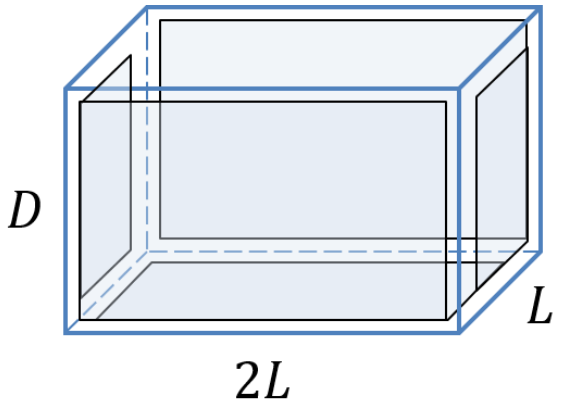
```
Solver status: ok
Termination condition: optimal
Optimal solution:
x1 = 42.8571428571429
x2 = 51.4285714285714
```

Introduction to Pyomo

Example 2:

$$\min \quad 1200(6DL + 2L^2)$$

$$s.t. \quad 2DL^2 = 8$$




```python
# Create the optimization model
model = pyo.ConcreteModel()

# Decision variables
model.D = pyo.Var(domain=pyo.NonNegativeReals)
model.L = pyo.Var(domain=pyo.NonNegativeReals)

# Objective Function
model.obj = pyo.Objective(expr=1200*(6*model.D*model.L + 2*model.L**2),
    sense=pyo.minimize)

# Constraints
model.con = pyo.Constraint(expr=2*model.D*model.L**2 == 8)

solver = pyo.SolverFactory("ipopt")
results = solver.solve(model, tee=False)

print("Solver status:", results.solver.status)
print("Termination condition:", results.solver.termination_condition)

print("Optimal solution:")
print(f"D = {pyo.value(model.D)}")
print(f"L = {pyo.value(model.L)}")
```

```
Solver status: ok
Termination condition: optimal
Optimal solution:
D = 1.2114137285548876
L = 1.8171205928320437
```

Outline

1. Motivational Example
2. Optimization Problems
3. Why is Optimization hard?
4. Overview of optimization solvers
5. Introduction to Pyomo